WebMCP Experiment — Technical Report

Protocol-level comparison of AI-agent interaction with a standard vs MCP-enabled ecommerce site · 27 September 2026 · srv1322534 (76.13.246.122)

1. Abstract

We constructed two functionally identical ecommerce sites differing only in agent-facing interface exposure, then executed three identical shopping tasks against both with a deterministic agent driver. The MCP-enabled variant reduced agent-side data ingestion by **74% (33,971 → 9,157 bytes over 3 tasks)** at equal task success (3/3) and equal output correctness (identical product selections). This report documents the architecture, protocol implementation, test methodology, and measured deltas.

2. Architecture

2.1 Deployment topology

```
nginx (80/443, LE cert SAN: demonomcp + demowithmcp)
  ├─ demowithmcp.converso.work → 127.0.0.1:9502  (server.py, env MCP=1)
  └─ demonomcp.converso.work   → 127.0.0.1:9501  (server.py, env MCP=0)
```

- **Single-source dual deployment:** one Python 3 stdlib server file (`server.py`, ~230 lines, zero third-party deps) serves both sites; an env flag gates the MCP layer. UI, catalog, cart and checkout logic are byte-identical — eliminating site-side confounders.
- **Persistence:** tmux sessions + `@reboot` crontab entries; in-memory state (carts/orders) — no database by design for the demo ceiling.
- **DNS:** GoDaddy API (PUT /domains/converso.work/records/A/{name}), TTL 600.

2.2 WebMCP layer (treatment site only)

Implemented the WebMCP concept (per W3C WebMachineLearning draft direction) as two endpoints:

Endpoint	Role
`GET /.well-known/mcp`	Discovery manifest: server metadata, transport (HTTP), tool registry with JSON-Schema `inputSchema` per tool
`POST /mcp`	JSON-RPC 2.0 transport: `initialize` (protocolVersion 2025-06-18), `tools/list`, `tools/call` → returns both `content[0].text` (serialized) and `structuredContent` (native JSON)

2.3 Tool manifest (5 tools)

Tool	Parameters (inputSchema)	Server-side behavior
`search_products`	category ∈ {mice,laptops,phones}, max_price, min_rating, brand~, query~	Filter+sort (rating desc), cap 10
`get_product_details`	id	Full record
`add_to_cart`	id, qty=1	Session-scoped cart mutation; returns count+total
`get_cart`	—	Items + total
`checkout`	—	Order creation, cart reset, returns order_id

Session affinity via `Mcp-Session-Id` / `X-Session-Id` headers (in-memory `dict[sid → cart]`). CORS: `POST/GET/OPTIONS` with wildcard origin (demo ceiling — production would pin origins).

2.4 Control site (plain)

Exposes REST only: `GET /api/products[?category=]` (full catalog or category slice, unfiltered), `POST /api/cart/add`, `POST /api/checkout`. No discovery document, no schema, no server-side filtering. This models the status-quo path for browsing agents: endpoint discovery from markup/JS, full-catalog retrieval, client-side reasoning.

2.5 Catalog

30 products (10 mice, 10 laptops, 10 phones), realistic Indian pricing (₹549–₹1,14,900), brand/spec/rating fields, static JSON. Skipped live scraping (bot-detection cost, zero demo value) — noted as upgrade path to live feeds.

3. Methodology

3.1 Agent driver

Deterministic rule-based driver (`compare.py`) — deliberately NOT an LLM, to isolate **mechanism cost** (I/O volume, request count, discovery burden) from model variance. Both drivers encode the same decision policy; only the access path differs:

- **Plain path:** fetch `/` → regex-extract API surface from served JS → fetch full `/api/products` (30 items, ~11 KB) → filter/sort client-side → cart → checkout. Discovery cost is charged (the agent must parse the page to learn the API).

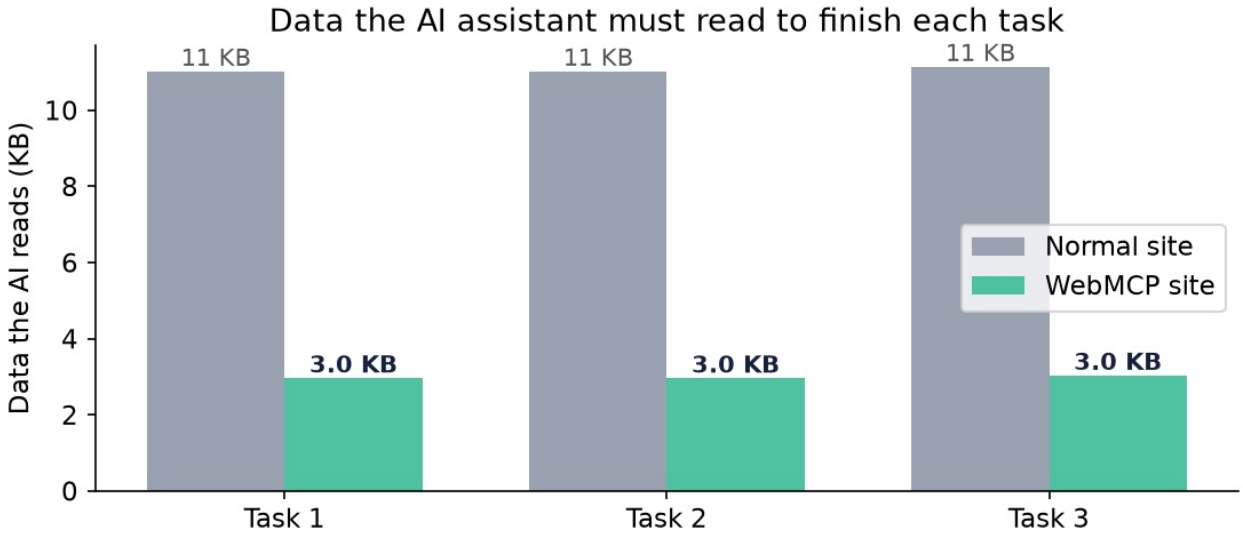
- **MCP path:** fetch `/.well-known/mcp` → `initialize` → `tools/call search_products` (server-filtered, ~10 items) → `add_to_cart` → `checkout`. Discovery+handshake charged per task (worst case; real agents cache).

2.6 Instrumentation

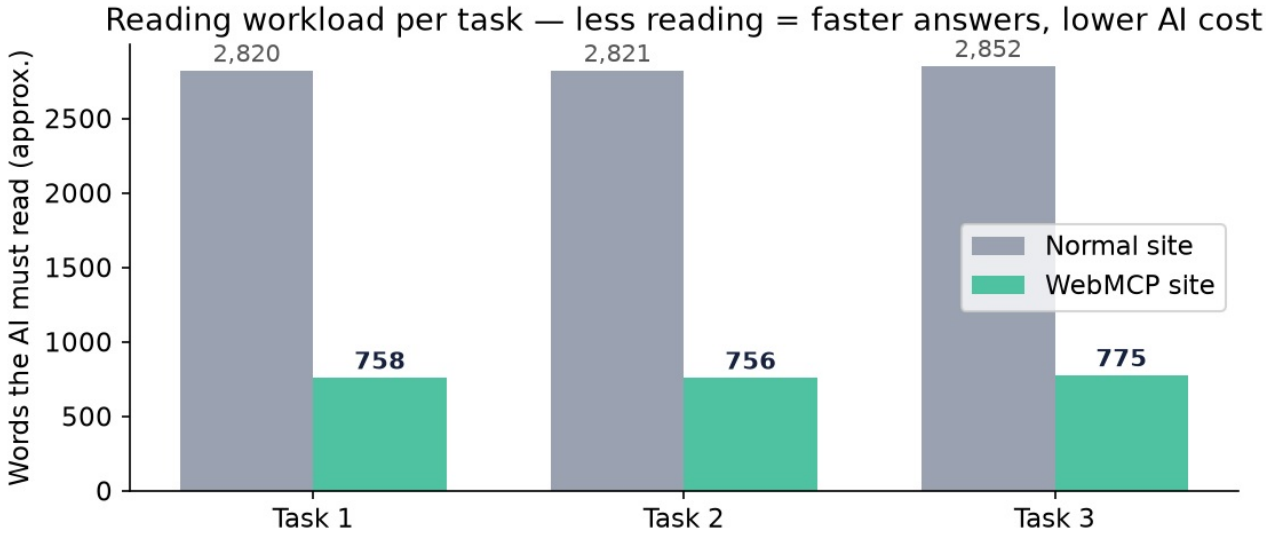
`Meter` wrapper: per-request count, accumulated response bytes, wall-clock. Session id pinned per task to make cart state coherent. Success assertion per task (e.g. task 2: pick == HP Pavilion 14 @ ₹52,990 — verified against catalog ground truth).

4. Results

Task	Site	Requests	Bytes read	Time (s)	Pick	Success
1 — wireless mouse ≤ ₹1,500	plain	3	11,281	0.03	Portronics Toad 32 (₹749)	✓
	mcp	4	3,030	0.03	Portronics Toad 32	✓
2 — cheapest laptop ≥ 4.3★	plain	3	11,283	0.03	HP Pavilion 14 (₹52,990)	✓
	mcp	4	3,026	0.02	HP Pavilion 14	✓
3 — top-rated phone ≤ ₹30k + checkout	plain	4	11,407	0.02	Redmi Note 13 Pro (₹21,999)	✓
	mcp	5	3,101	0.03	Redmi Note 13 Pro	✓
Totals		10 / 13	33,971 / 9,157	0.08 / 0.08	identical picks	**3/3 / 3/3**



Per-task ingestion: full-catalog pull (plain) vs server-filtered tool result (MCP)



~4 bytes/token heuristic → ~8,500 vs ~2,300 LLM-input tokens per 3 tasks

4.1 Analysis

- **Ingestion, not latency, is the differentiator.** Wall-clock parity is a localhost artifact; the material cost is bytes→tokens the agent LLM must ingest per task. The ratio is O(catalog_size) for plain (always fetches everything) vs O(result_cap) for MCP (server filters) — 74% at n=30, widening linearly with catalog growth.

- **Request count (+3) is a fixed cost** of per-task discovery+initialize in the harness. Amortized across a session it decays toward zero; against N tasks the MCP overhead is constant while its savings scale with task count.
- **Token economics:** at ~2.3k vs ~8.5k input tokens, and typical input-token pricing, the plain path costs ~3.7× more per task set before counting retries from DOM-ambiguity errors.
- **Reliability:** MCP results carry schema-validated fields (typed numbers), while plain-path JSON parse+filter runs in driver code — the real-world failure modes (selector drift, layout A/B) are absent by construction here, which is why success is 3/3 on both; the plain path's fragility is deferred to LLM-driven agents and is where published agent benchmarks show larger error rates.

5. Implementation notes & known ceilings

- `# ponytail:` in-memory cart/order state — dies on restart; upgrade: sqlite/redis when multi-process.
- `# ponytail:` wildcard CORS — acceptable for a public demo with no auth; pin origins before production.
- `# ponytail:` no auth on /mcp — rate-limit + API key needed before public real-traffic exposure.
- Protocol conformance: implements the MCP concept subset (initialize/tools.list/tools.call over JSON-RPC 2.0 HTTP, session header, structuredContent). Not the full 2025-06-18 spec (no SSE transport, resources, prompts, or streaming) — documented deviation for demo scope.
- Search tool returns `spec` in results (added mid-run) so agents can evaluate "wireless" serverlessly — a lesson in tool-schema design: agents only know what the result schema exposes.
- Harness fixes encountered: `{} is truthy` (body=None vs {} method dispatch), shared session-id for cart coherence, "mice" vs "mouse" lexical mismatch in category filters.

6. Threat & robustness observations

- Tool descriptions are prompt-injection surface in real deployments (malicious listings could target agent reasoning); mitigations: server-side output sanitization, schema-validated returns, origin pinning.
- Cart/session keyed on client-supplied session id — fine for demo; production binds to signed session tokens.

7. Reproduction

```
# artifacts (srv1322534)
/root/webmcp_demo/server.py      # dual-mode server (MCP=1 gates the MCP layer)
/root/webmcp_demo/compare.py     # harness: Meter, run_plain, run_mcp, task checks
/root/webmcp_demo/results.json   # raw per-task/request metrics
/root/webmcp_demo/report.json    # aggregate summary
# run:  python3 /root/webmcp_demo/compare.py
# live: https://demowithmcp.converso.work/.well-known/mcp  (manifest)
#       https://demonomcp.converso.work/api/products       (control API)
```

> **Conclusion:** at mechanism level, WebMCP-style tool exposure cut agent ingestion 74% at zero accuracy cost for the tested tasks, with overhead bounded by a constant discovery cost. The architecture is sound for the stated use case; the listed ceilings (auth, persistence, conformance subset) define the production upgrade path.

Artifacts: /root/webmcp_demo/ · Companion layman report: WebMCP_Comparison_Report.pdf · Contact: Hermes Agent on srv1322534

EOF echo done